0 6 2

fixtures/render_keys/ — T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/docs/render_keys.py` (plan.md T-C12 "Content-addressed, parallel, batched twin regeneration"; SC-002, FR-022 in `specs/004-fast-dev-cycles/spec.md`). No `contracts/` file names this tool explicitly (confirmed absent by this file's own author, per the same "no dedicated contract exists" pattern T057/T060/T061 already document for their own tools — `specs/004-fast-dev-cycles/ contracts/` has no `render-keys*.md` or `twin-rendering*.md` entry). Task T062 (`tasks.md`) writes ONLY this RED test + these fixtures; the tool itself is a later, separate task (Producer != Verifier, §11.4.240 — this file's author never implements `render_keys.py`).

Plan.md T-C12's own Work + Protecting-tests lines, verbatim:

> `docs/render_keys.py` stores per twin `key = hash(source bytes ‖ stylesheet ‖ exporter version ‖ pandoc/weasyprint versions ‖ format)` ; `sync_all_markdown_exports.sh` skips a twin whose key matches, renders changed twins in parallel within the §12.6 cap, and renders once per commit for all touched sources; the freshness gate checks the key (a skipped render is still proven current), in addition to mtime.
>
> Protecting tests: RED: an unchanged source re-renders today; golden: a changed source re-renders all four formats; golden-bad: a twin whose source changed but whose key was not updated is caught by the freshness gate; paired mutation: key on mtime only → a touched-but-identical file fixture shows the difference; a stale twin fixture FAILs the gate.

tasks.md T062's own line folds the paired-mutation clause's fixture requirement into this task's own scope: "(an unchanged source re-renders today; a changed source re-renders all four formats; golden-bad: a twin whose source changed but whose key was not updated is caught by the freshness gate; touched-but-identical file fixture shows mtime vs content difference)". The full paired-mutation FLIP itself (T063's job, run once T-C12's real tool lands) reuses these same 4 fixtures.

The tool under test (not yet implemented — T-C , a later task)

`constitution/scripts/fastcycle/docs/render_keys.py`, per plan.md's path table (line 196: `render_keys.py # content-addressed twin rendering (T-C12)`). `constitution/scripts/fastcycle/docs/` currently holds nothing but `.gitkeep` — confirmed by this RED test's own Section A control needle, never assumed.

Assumed CLI contract (UNCONFIRMED by any contract file — DEFINED here for this RED test's own GREEN-branch, binding-if-adopted on T-C12's implementer, following the house precedent in `fixtures/verdict_cache/README.md`, `fixtures/dispatch_stamp/`, and `test_batch_bisect_red.sh`'s own "no named contract file existed" precedent):

```
render_keys.py compute --source <path.md> --format {html|pdf|docx} \
    --stylesheet <path> --exporter-version <str> \
    --pandoc-version <str> --weasyprint-version <str>
```

Computes the per-twin key from the CURRENT source bytes + the named components, per T-C12's Work-line formula, and prints the 64-hex-char SHA-256 key on stdout. Exit 0 on success.

```
render_keys.py check --source <path.md> --format {html|pdf|docx} \
    --key-file <path/to/key.json> \
    --stylesheet <path> --exporter-version <str> \
    --pandoc-version <str> --weasyprint-version <str>
```

Reads the RECORDED key from `--key-file` (a JSON sidecar shaped like `key_v1.json` / `key_matching.json` below — at minimum a top-level `"key"` field), recomputes the CURRENT key from the same 5 flags read live, and compares. On a match, prints `FRESH` and exits 0 (the exporter may safely skip the render — "a skipped render is still proven current", per T-C12's own Work line). On a mismatch, prints `STALE` (naming the differing component where determinable) and exits non-zero — this IS "the freshness gate" T-C12's Work line and `rk_stale_key_caught/` 's golden-bad scenario both name.

```
render_keys.py store --source <path.md> --format {html|pdf|docx} \
    --key-file <path/to/key.json> \
    --stylesheet <path> --exporter-version <str> \
    --pandoc-version <str> --weasyprint-version <str>
```

Computes the key from the CURRENT inputs (same formula as `compute`) and writes/overwrites `--key-file` with the recorded key. Exit 0 on success.

T-C12's implementer MAY choose a different flag/subcommand shape — if so, per the same precedent, that choice belongs in `render_keys.py` 's own docstring, and this RED test's fixtures should then be read as "the semantic scenario", with the concrete CLI invocation lines adapted (never as an unreviewable, silent contract override).

The key formula this fixture set actually uses

A standalone reference implementation, `tests/lib/render_keys_ref.py` (never `render_keys.py` itself — see its own module docstring for the full disclaimer, mirroring `tests/lib/dec07_key_ref.py` 's), computes:

```
key = SHA-256(
  source bytes ||
  stylesheet (path/identifier string) ||
  exporter version (a SHA-256 of scripts/testing/
sync_all_markdown_exports.sh's
    OWN bytes at fixture-authoring time — the exporter publishes no
    semver today, so this content hash is the defensible, content-
addressed
    proxy this fixture set adopts, mirroring DEC-07's own "gate-script
    bytes are a key component" precedent) ||
  pandoc version string (`pandoc --version`'s first line) ||
  weasyprint version string (`weasyprint --version`) ||
  format ("html" | "pdf" | "docx")
)
```

Field separator: U+001F (unit separator), matching `dec07_key_ref.py`'s own convention — reused for consistency, not independently re-derived. Every key embedded in `key_v1.json` / `key_matching.json` below was computed by a REAL run of `render_keys_ref.py` against this fixture set's own REAL source files and this HOST's REAL, captured `pandoc --version` / `weasyprint --version` output at fixture-authoring time (2026-09-29) — a control-needle self-check (determinism: two computations of the same inputs agree; non-vacuous: a different `format` argument, and a one-byte source change, each yield a different key) was run and confirmed before either JSON file was written (§11.4.201(7)(a)/§11.4.273).

The scenarios

Directory	Task-line clause	Kind	What the test does
`rk_unchanged_today/`	"an unchanged source re-renders today"	control needle	`touch`es a real toy `source.md` (content byte-identical, sha256-verified before/after) and shows a REAL, non-`--check-only` invocation of the CURRENT `scripts/testing/sync_all_mark-down_exports.sh` re-renders its already-rendered `.html` / `.pdf` / `.docx` siblings anyway (their mtimes bump again) — because today's freshness check is mtime-only (`[ "$md" -nt "$html" ]`), proven by grepping the real exporter's own source text.
`rk_changed_all_four/`	"a changed source re-renders all four formats"	golden	Genuinely edits a toy `source.md`'s content (marker V1 → V2), and shows a REAL invocation of the CURRENT exporter regenerates ALL THREE sibling formats (`.html` / `.pdf` / `.docx`) — mtime-bumped AND content-updated (new marker present, old marker gone) — alongside the `.md` itself: the full §11.4.65 four-format set.

Directory	Task-line clause	Kind	What the test does
`rk_stale_key_caught/`	"golden-bad: a twin whose source changed but whose key was not updated is caught by the freshness gate"	golden-bad	Genuinely edits `source.md` (V1 → V2) WITHOUT updating `key_v1.json` (which still records V1's key), then genuinely invokes the (today, absent) `render_keys.py check` against this stale pairing — real invocation, rc=127 today, self-flips GREEN (expected: `STALE` + non-zero exit) once T-C12 lands.
`rk_touched_identical/`	"touched-but-identical file fixture shows mtime vs content difference"	self-validation + golden	`touch`es `source.md` (NEVER edits its bytes — sha256-verified identical before/after, a real measured mechanism proof), then genuinely invokes the (today, absent) `render_keys.py check` against `key_matching.json` (which DOES match the current, unedited content) — real invocation, rc=127 today, self-flips GREEN (expected: `FRESH` + exit 0) once T-C12 lands. Direct positive contrast to `rk_unchanged_today/`'s proof of today's wasteful mtime-only behaviour on the exact same class of input.

Every scenario's `source.md` is restored to its ORIGINAL checked-in bytes, and every siblings/generated file (`.html` / `.pdf` / `.docx`) this test itself renders is removed, on every exit path (`trap ... EXIT`) — so the tracked fixture tree always returns to `source.md` (+ `key_*.json` / `expected.json` where present) only, matching the established convention across every other `fixtures/*/` directory in this tree (no generated binary siblings committed). Re-running the test is therefore safe and idempotent.